Chapter - 3 Lab-Assignment

Name : Bhesaniya Kruti

Enrollment no. : 230801059

Roll no. : 07

Program : BCA

Subject : PHP

1. Create a folder named classes/ with the following files:
    - Post.php (defines a Post class)
    - Author.php (defines an Author class)
    - Category.php (defines a Category class)
    - Use spl_autoload_register() in index.php to automatically load the required class files from the classes/ folder.
    - Instantiate objects of Post, Author, and Category classes without manually including their files.

Answer→

Post.php :

```
<?php
class Post{
    public function showPost(){
        echo"Post Name: Author ";

    }
}

?>
```

Author.php :

```
<?php
class Author{

    public function showAuthor(){
        echo"Author Name: james clear";
    }

}
```

```
?>
```

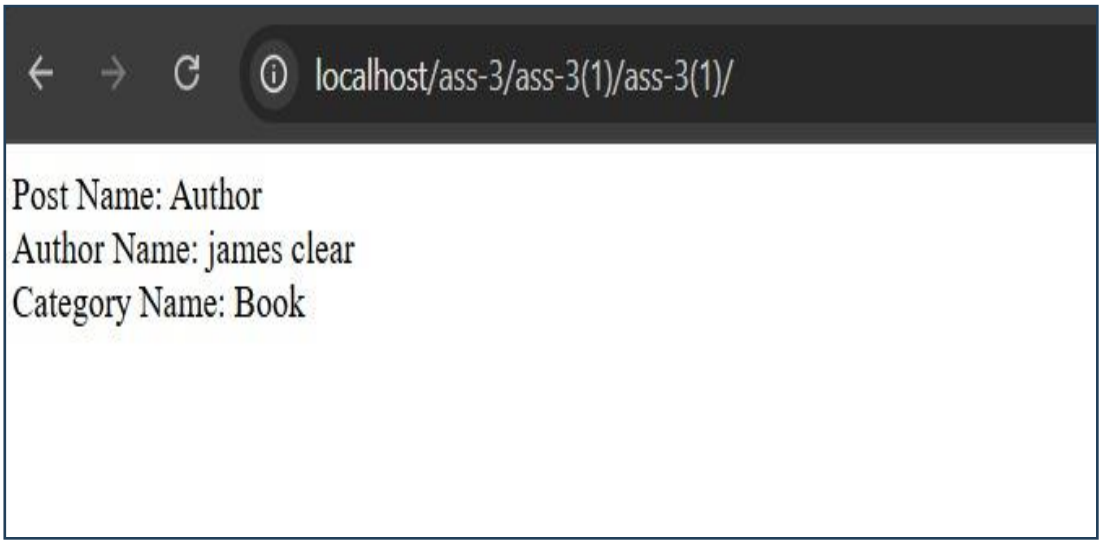
Category.php :

```
<?php
class Category{
    public function showCategory(){
        echo"Category Name: Book ";

    }
}

?>
```

index.php :

```
<?php
spl_autoload_register(function($index)
{
    include $index.".php";
});

$p=new Post();
$p->showPost();
echo "<br>";

$a=new Author();
$a->showAuthor();
echo "<br>";

$c=new Category();
$c->showCategory();
?>
```

Output→



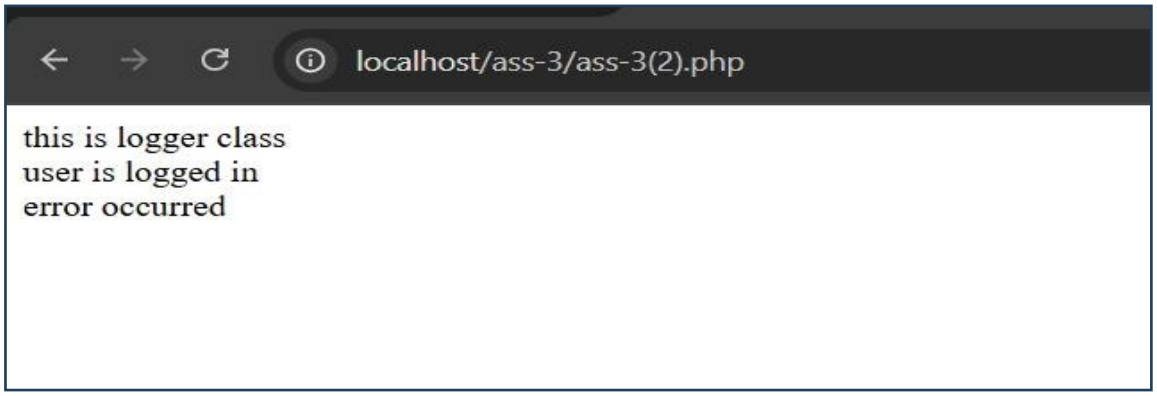
2. Create a base class called Logger with a method logMessage($msg).
   - Use an anonymous class that extends Logger to override the logMessage() method — e.g., to prefix messages with a type:
   - For user log: [USER] ...
   - For error log: [ERROR] ...
   - Create and use multiple anonymous class objects for different log types, each overriding the method behavior differently.
   - Call logMessage() for each anonymous object and show the output.

Answer→

```
<?php
  class Logger
  {
    public $msg;
    public function logMessage($msg)
```

```
    {
        return $this->msg = $msg;
    }
}




$user = new class extends Logger{
    public $msg;
    public function logMessage($msg)
    {
        return  $this->msg = $msg;
    }
};

$error = new class extends Logger{
    public $msg;
    public function logMessage($msg)
    {
        return $this->msg = $msg;
    }
};

$Log = new Logger();
echo $Log->logMessage("this is logger class");
echo "<br>";
echo $user->logMessage("user is logged in");
echo "<br>";
echo $error->logMessage("error occurred");

?>
```

Output→



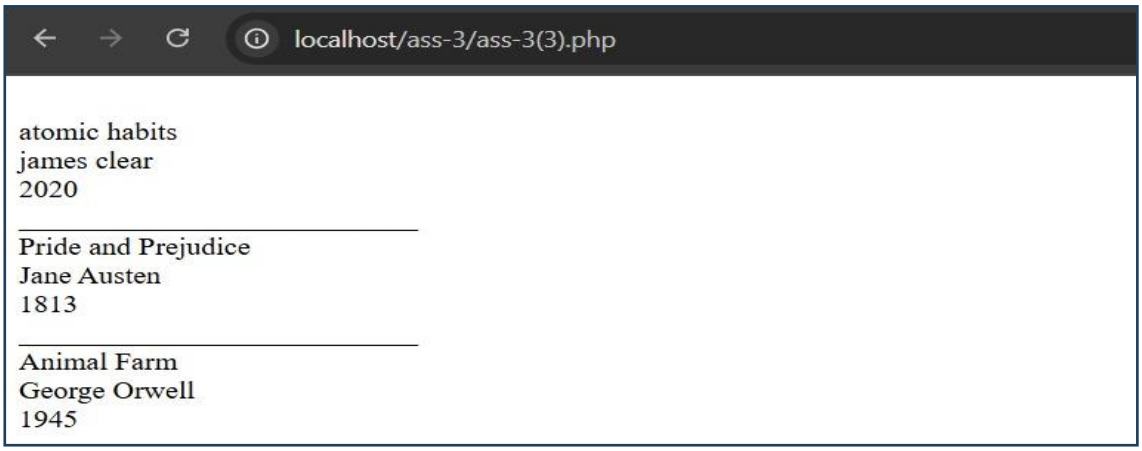
3. Create a book class with properties: title, author, year.
   - Create a Library class that:
   - Stores multiple Book objects.
   - Implements PHP's built-in Iterator interface (or uses IteratorAggregate) to allow iteration using foreach.
   - Add multiple books to the library.
   - Use a foreach loop to print each book's details through the Library object directly.

Answer→

```
<?php
class Library
{
    public $title,$author,$year;

    public function book($title,$author,$year){
        $this->title = $title;
        $this->author = $author;
        $this->year = $year;
    }

}
```

```
$b1 = new Library();
$b1->book("atomic habits", "james clear", 2020);

foreach($b1 as $key=>$value){
    echo "<br>" . $b1->title;
    echo "<br>" . $b1->author;
    echo "<br>" . $b1->year;
    echo "<br>_____________________________" ;
    echo "<br>";

}
?>
```

Output→



4. Create a Product class with properties: name, price, and category.

- Add a method displayProduct() to show product details.
- Use PHP's clone keyword to create a copy of an existing product object.

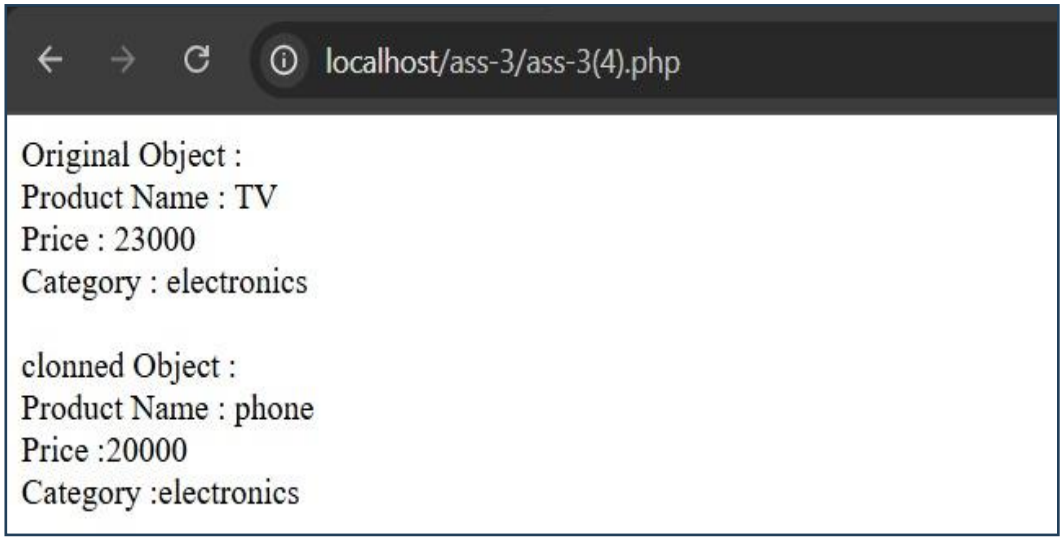
- Modify one or two properties in the cloned object and display both the original and the cloned product to verify they are independent.
- Optionally, implement the __clone() magic method to reset or update some properties automatically during cloning (e.g., assign a new SKU or set a clonedFrom label).

Answer→

```
<?php
 class Product{
   public $name,$price,$category;
   public function displayProduct($name,$price,$category){
      return "Product Name : ".$this->name=$name
      ."<br>". "Price : ".$this->price=$price
      ."<br>". "Category : ".$this->category=$category;

   }
 }

 $p1=new Product();
 echo "Original Object :<br>";
echo  $p1->displayProduct("TV",23000,"electronics");



 $p2=clone $p1;
 $p2->name="phone";
 $p2->price=20000;
 $p2->category="electronics";

echo "<br><br>";
```

```
 echo "clonned Object : <br>Product Name : ".$p2->name. "<br>Price :".$p2->price.
"<br>Category :".$p2->category;

 ?>
```

Output→

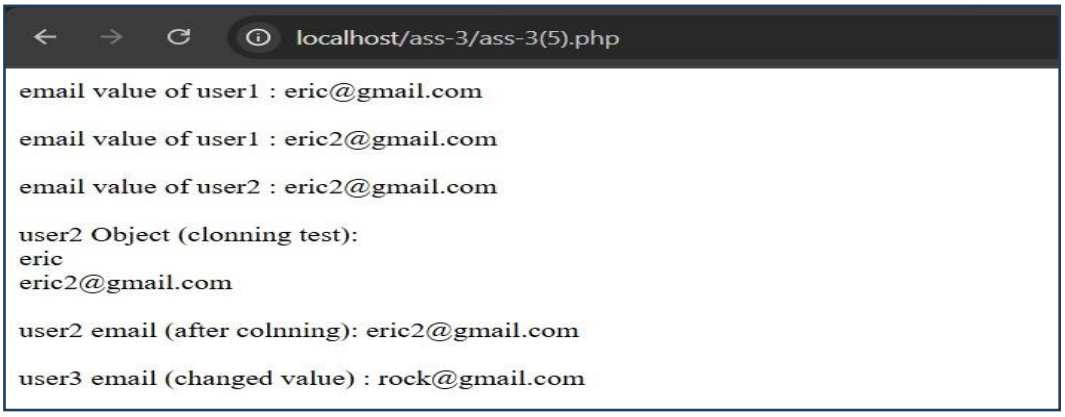


5. Create a class UserProfile with properties: username, email.

- Create an object $user1 and assign values to its properties.
- Assign $user1 to another variable $user2.
- Modify the email using $user2 and print the values using both $user1 and $user2.
- Observe and explain the output:
- Are both objects showing the same data?
- Did changing one affect the other?
- (Optional): Use clone to create an independent copy of the object and repeat the test.

Answer→

```
<?php
class userProfile
{
   public $username="eric";
   public $email="eric@gmail.com";
```

```
}

$user1=new userProfile();

$user2=$user1;
echo "email value of user1 : ".$user1->email;
$user2->email="eric2@gmail.com";

echo "<br><br>";
echo "email value of user1 : " . $user1->email;
echo "<br><br>";

echo "email value of user2 : ".$user2->email;
echo "<br><br>";

echo"user2 Object (clonning test): <br>". $user2->username."<br>".$user2-
>email;


$user3= clone $user2;
$user3->email="rock@gmail.com";

echo "<br><br>";
echo "user2 email (after colnning): ".$user2->email;
echo "<br><br>";
echo "user3 email (changed value) : ".$user3->email;

?>
```

Output→